

1. El valor None

En Python, None es un valor especial y una palabra clave que representa la **ausencia de un valor** o un valor nulo. No es lo mismo que 0 (cero), False (falso) o una cadena vacía "". Es un tipo de dato propio llamado NoneType.

Características clave

- **Es único:** Esto significa que solo existe una instancia de None en todo tu programa.
- **Comparación:** Para verificar si una variable es None, la mejor práctica es usar el operador de identidad is, no el de igualdad ==.

Usos comunes

1. **Inicialización:** Para declarar una variable que aún no tiene un valor real.
2. **Argumentos por defecto:** Como valor predeterminado para parámetros opcionales en funciones.
3. **Retorno implícito:** Si una función no tiene una sentencia return, Python devuelve None automáticamente.

Ejemplo de uso de None:

```
variable = None
```

```
if variable is None:
```

```
    print("La variable no tiene valor aún.")
```

```
# Diferencia con otros valores "vacíos"
```

```
print(None == 0)    # False
```

```
print(None == False) # False
```

```
print(None == "")    # False
```

2. Alcance de Funciones

El "alcance" se refiere a la región del código donde una variable es visible y accesible. Python resuelve los nombres de las variables buscando en cuatro niveles de alcance, conocidos por la regla **LEGB**.

Los niveles de alcance (LEGB)

1. **L (Local):** Variables definidas *dentro* de una función. Solo existen mientras la función se ejecuta y no son accesibles desde fuera.
2. **E (Enclosing):** En funciones anidadas, es el alcance de la función que "envuelve" a la actual.
3. **G (Global):** Variables definidas en el nivel superior del script o módulo. Son accesibles

desde cualquier lugar (para lectura), pero requieren la palabra clave global para ser modificadas dentro de una función.

4. **B (Built-in):** Nombres preasignados en Python (print, len, None).

Ejemplo:

```
x = 10 # Variable Global
```

```
def mi_funcion():  
    y = 5 # Variable Local  
    print(f"Dentro: x={x}, y={y}") # Puede leer la global 'x'
```

```
mi_funcion()  
# print(y) # Error! 'y' no existe fuera de la función
```

Modificar una variable global

Si intentas cambiar x dentro de la función sin avisar a Python, crearás una nueva variable local llamada x en lugar de cambiar la global. Para cambiar la global, usas global:

```
contador = 0
```

```
def incrementar():  
    global contador # Avisamos que usaremos la variable de fuera  
    contador += 1
```

```
incrementar()  
print(contador) # Imprime 1
```

Relación entre None y Funciones

Si defines una función y no escribes explícitamente qué debe devolver (usando return), Python asume implícitamente que el resultado es None.

Ejemplo:

```
def saludo(nombre):  
    print(f"Hola, {nombre}")  
    # No hay 'return' aquí
```

```
resultado = saludo("Ana")
```

```
print(resultado)  
# Salida: None
```